

Universidad Politécnica de San Luis Potosí

**LAB: FILE PATH TRAVERSAL TRAVERSAL SEQUENCES
BLOCKED WITH ABSOLUTE PATH BYPASS**

Presentado por:

Christian Alfredo Morales Meza - 184937

Asignatura: Fundamentos de Ciberseguridad

Profesor: Servando López Contreras

Fecha de entrega: 09/09/2026

Índice de contenido

Índice de contenido	2
Ficha del laboratorio	3
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento paso a paso.....	4
Evidencia del procedimiento.....	4
Evidencias obligatorias	9
Explicación del payload.....	10
Mitigación recomendada.....	11

Ficha del laboratorio

Laboratorio: LAB: LAB: FILE PATH TRAVERSAL TRAVERSAL SEQUENCES BLOCKED WITH ABSOLUTE PATH BYPASS

Nivel: Apprentice

Tipo de explotación: Path Traversal mediante evasión con ruta absoluta (Direct/Absolute Path Traversal)

Objetivo del laboratorio

Recuperar y visualizar el contenido del archivo sensible `/etc/passwd` del sistema operativo, evadiendo el filtro de secuencias de salto relativo mediante el uso de una ruta absoluta en el parámetro `filename`.

Explicación técnica de la vulnerabilidad

La vulnerabilidad de Path Traversal (o salto de directorio) se origina cuando una aplicación web incorpora entradas controladas por el usuario directamente en funciones y llamadas del sistema de archivos (como `open()`, `readFile()` o `fopen()`) sin la debida validación sintáctica ni canonización de la ruta.

En este escenario específico, la causa raíz radica en una implementación defectuosa de la defensa basada en listas negras o patrones de bloqueo. Los desarrolladores configuraron un filtro que detecta e invalida secuencias de navegación relativa convencionales (como `'..'` y `'..\'`). No obstante, el backend procesa el valor del parámetro `'filename'` de manera tal que, si recibe una ruta absoluta que inicia desde la raíz del sistema de archivos (`'/'`), la API de lectura de archivos del sistema operativo (en este caso Linux/Unix) la interpreta directamente como una ubicación raíz global, ignorando el directorio base predeterminado de almacenamiento de imágenes (`/var/www/images/`).

Dentro de las variantes técnicas de Path Traversal, este vector corresponde a una evasión directa por ruta absoluta (Absolute Path Bypass), a diferencia de ataques basados en secuencias anidadas (`.../`), codificación URL múltiple o inyección de bytes nulos (`%00`).

En el marco OWASP Top 10, se clasifica rigurosamente bajo 'A01:2021 - Broken Access Control'. Respecto a la tríada CIA:

- Confidencialidad: Impacto crítico. Permite la exfiltración no autorizada de archivos del sistema operativo, código fuente de la aplicación, credenciales en texto plano, tokens de configuración y variables de entorno.
- Integridad: Nulo en lectura pura, pero si la misma debilidad existiera en un flujo de escritura o carga de archivos, permitiría la sobreescritura de binarios o bibliotecas críticas.
- Disponibilidad: Nulo a bajo, salvo que se lean archivos especiales bloqueantes del sistema o se consuman recursos excesivos.

En un entorno corporativo real, esta brecha expone la estructura de cuentas locales (`/etc/passwd`), claves privadas SSH, llaves de API y archivos de conexión a bases de datos, facilitando una escalada de privilegios o el movimiento lateral del atacante.

Procedimiento paso a paso

Inicialmente, se configuró el entorno de auditoría iniciando el proxy de Burp Suite Community Edition y abriendo el navegador integrado para interceptar el tráfico web generado por la aplicación.

Se navegó a la página principal del laboratorio de comercio electrónico. Al inspeccionar el renderizado de la tienda, se observó que las imágenes de los productos se obtienen mediante llamadas al endpoint '/image' pasando el nombre del recurso a través del parámetro 'filename' (por ejemplo, 'GET /image?filename=42.jpg'). Esta petición fue localizada en el historial HTTP de Burp Proxy y enviada a la pestaña Repeater para su análisis interactivo.

En Repeater, se ejecutó inicialmente una prueba de concepto estándar inyectando secuencias de navegación relativa '../' repetidas hacia el archivo de usuarios del sistema: 'filename=../../../../etc/passwd'. El servidor respondió con un código de estado 'HTTP/2 400 Bad Request' y el mensaje '"No such file"', evidenciando la presencia de un mecanismo de validación que bloquea o neutraliza dichas secuencias.

Posteriormente, con base en la hipótesis de que la aplicación trata las entradas como rutas directas y únicamente filtra secuencias relativas, se sustituyó el valor por la ruta absoluta directa del archivo: '/etc/passwd'. Al enviar la solicitud, el servidor devolvió un código 'HTTP/2 200 OK' junto con el volcado completo del archivo '/etc/passwd' en el cuerpo de la respuesta. Finalmente, al refrescar la interfaz de la tienda en el navegador, se constató la resolución exitosa del laboratorio.

Evidencia del procedimiento

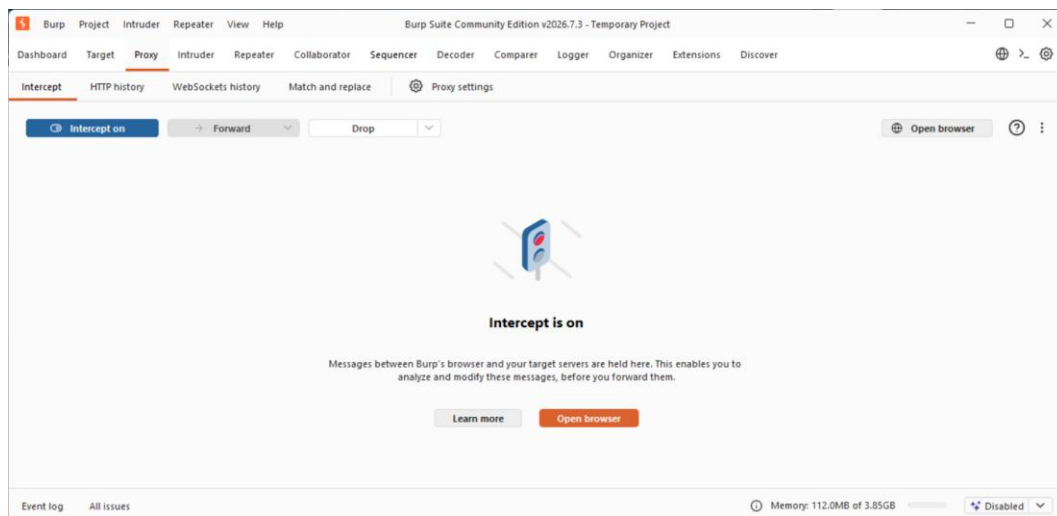


Figura 1. Activación de la interceptación de tráfico en el proxy de Burp Suite y apertura del navegador integrado.

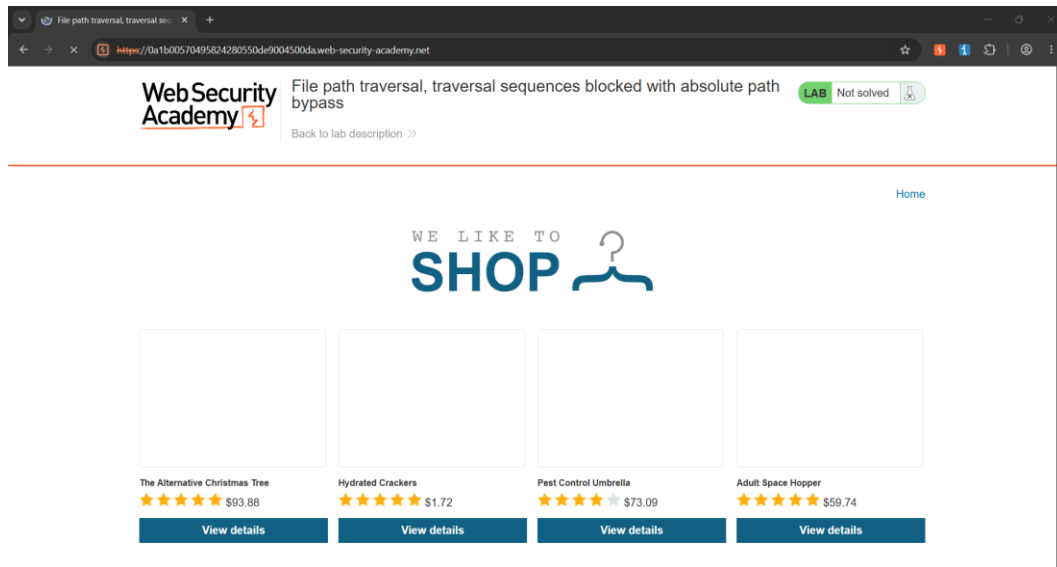


Figura 2. Navegación inicial hacia el catálogo de productos en la interfaz web del laboratorio.

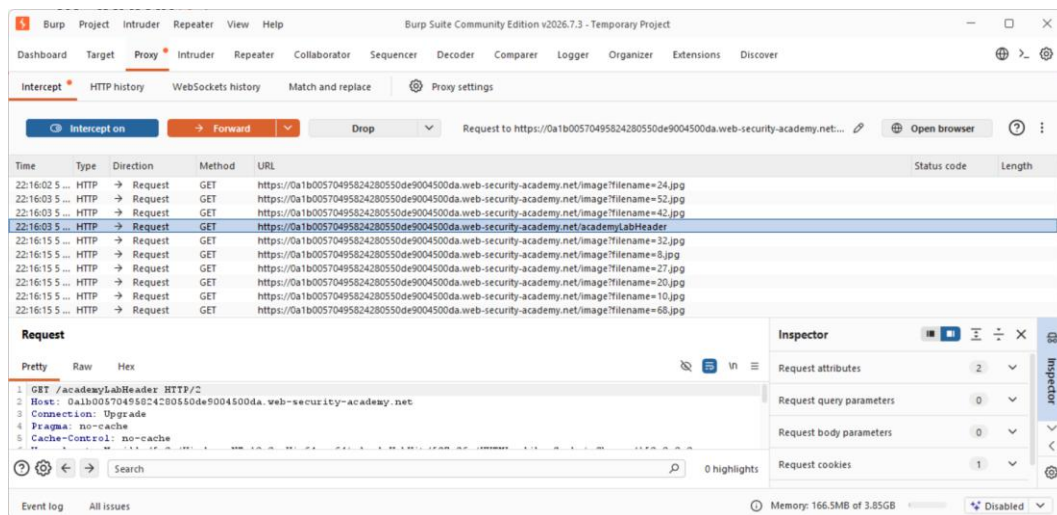


Figura 3. Captura en tiempo real de las solicitudes HTTP asociadas a la carga de activos de la página.

The screenshot shows the Burp Suite HTTP history tab. The table lists various HTTP requests. The selected request is a GET request to `https://0a1b00570495d24260.../image?filename=42.jpg`. The request details are visible in the bottom pane, showing the raw HTTP request with headers like `Host: 0a1b00570495d24260.../image?filename=42.jpg` and `Accept: */*`.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response...
43	https://0a1b00570495d24260...	GET	/image?filename=42.jpg		✓	200	98514	JPEG				✓	79.125.84.16		22:19:33 S...	8080	439
8	https://0a1b00570495d24260...	GET	/image?filename=52.jpg		✓	200	143243	JPEG				✓	79.125.84.16		22:16:03 S...	8080	237
42	https://0a1b00570495d24260...	GET	/image?filename=52.jpg		✓	200	142243	JPEG				✓	79.125.84.16		22:19:33 S...	8080	437
25	https://0a1b00570495d24260...	GET	/image?filename=56.jpg		✓	200	163137	JPEG				✓	79.125.84.16		22:16:15 S...	8080	1003
57	https://0a1b00570495d24260...	GET	/image?filename=56.jpg		✓	200	163137	JPEG				✓	79.125.84.16		22:19:40 S...	8080	213
22	https://0a1b00570495d24260...	GET	/image?filename=9.jpg		✓	200	118082	JPEG				✓	79.125.84.16		22:16:15 S...	8080	1277
54	https://0a1b00570495d24260...	GET	/image?filename=6.jpg		✓	200	118082	JPEG				✓	79.125.84.16		22:19:39 S...	8080	208
21	https://0a1b00570495d24260...	GET	/image?filename=6.jpg		✓	200	83202	JPEG				✓	79.125.84.16		22:16:15 S...	8080	1313
53	https://0a1b00570495d24260...	GET	/image?filename=6.jpg		✓	200	83202	JPEG				✓	79.125.84.16		22:19:38 S...	8080	209
15	https://0a1b00570495d24260...	GET	/image?filename=4.jpg		✓	200	166301	JPEG				✓	79.125.84.16		22:16:15 S...	8080	684
47	https://0a1b00570495d24260...	GET	/image?filename=4.jpg		✓	200	166301	JPEG				✓	79.125.84.16		22:19:36 S...	8080	334
31	https://0a1b00570495d24260...	GET	/image?filename=9.jpg		✓	200	166611	JPEG				✓	79.125.84.16		22:16:15 S...	8080	258
43	https://0a1b00570495d24260...	GET	/image?filename=9.jpg		✓	200	166611	JPEG				✓	79.125.84.16		22:19:42 S...	8080	216
4	https://0a1b00570495d24260...	GET	/resources/css/habocommerce.css		✓	200	28347	CSS	css			✓	79.125.84.16		22:16:02 S...	8080	222
37	https://0a1b00570495d24260...	GET	/resources/css/habocommerce.css		✓	200	28347	CSS	css			✓	79.125.84.16		22:19:40 S...	8080	213
16	https://0a1b00570495d24260...	GET	/resources/images/voting1.png		✓	200	708	PNG	png			✓	79.125.84.16		22:16:15 S...	8080	215
48	https://0a1b00570495d24260...	GET	/resources/images/voting1.png		✓	200	708	PNG	png			✓	79.125.84.16		22:19:39 S...	8080	208

Figura 4. Identificación de la petición GET dirigida al recurso /image con el parámetro filename en el historial HTTP.

The screenshot shows the same Burp Suite HTTP history tab. A context menu is open over the selected request (GET /image?filename=42.jpg), showing options like 'Send to Repeater', 'Send to Sequencer', 'Send to Intruder', 'Send to Repeater (request)', 'Send to Sequencer (request)', 'Send to Intruder (request)', 'Send to Repeater (response)', 'Send to Sequencer (response)', 'Send to Intruder (response)', 'Open response in browser', 'Request in browser', 'Engagement tools (Pro version only)', 'Show new history window', 'Add notes', 'Highlight', 'Delete item', 'Clear history', 'Copy URL', 'Copy as curl command (bash)', 'Copy links in response', 'Save item', and 'Proxy history documentation'.

Figura 5. Envío de la solicitud de obtención de imagen al módulo Repeater para pruebas de manipulación.

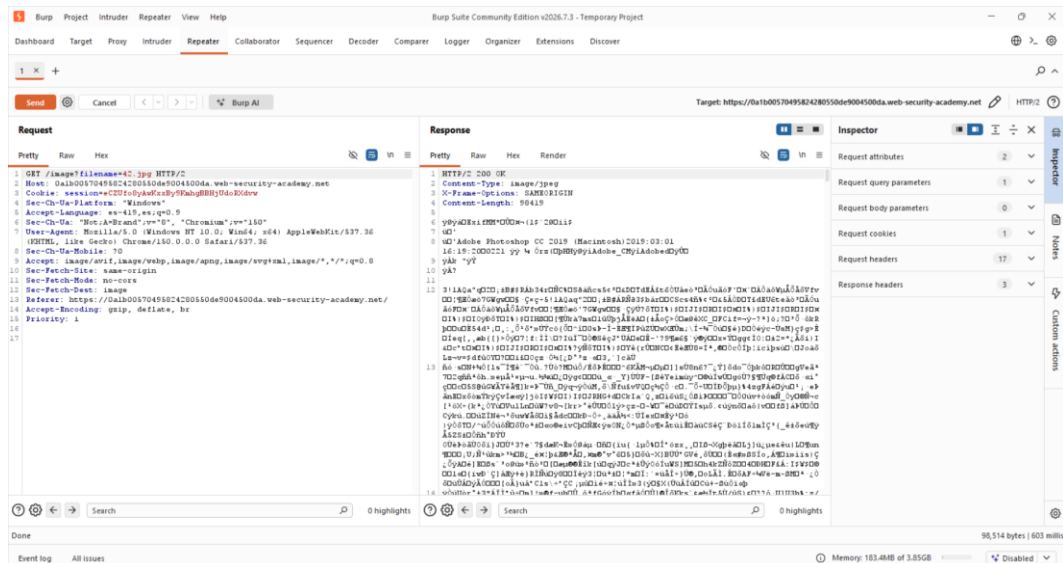


Figura 6. Envío inicial de la petición en Repeater confirmando la recepción exitosa de los bytes de la imagen original.

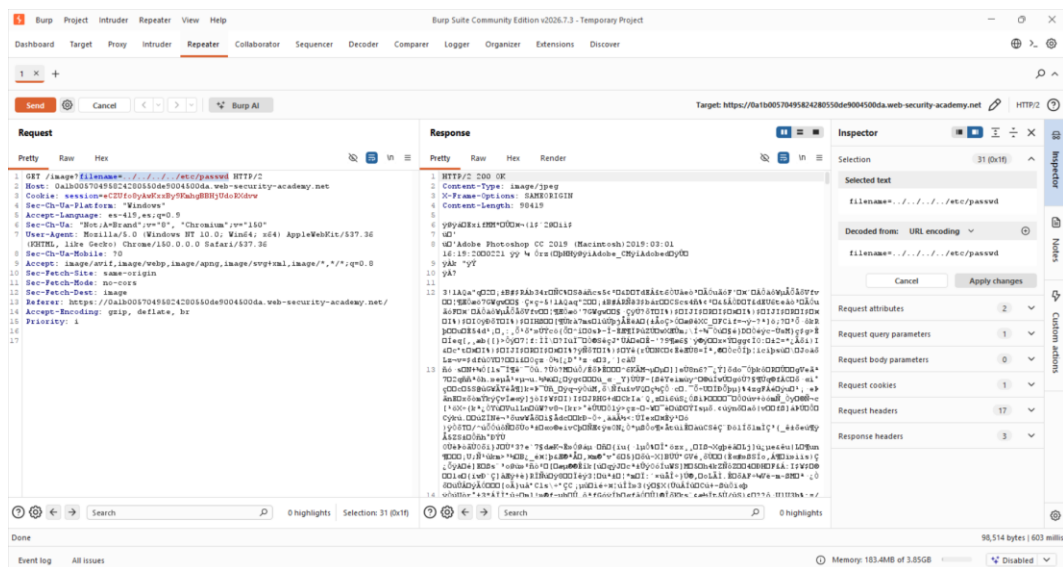


Figura 7. Inyección del payload con secuencias de salto relativo `../../../../etc/passwd` en el parámetro filename.

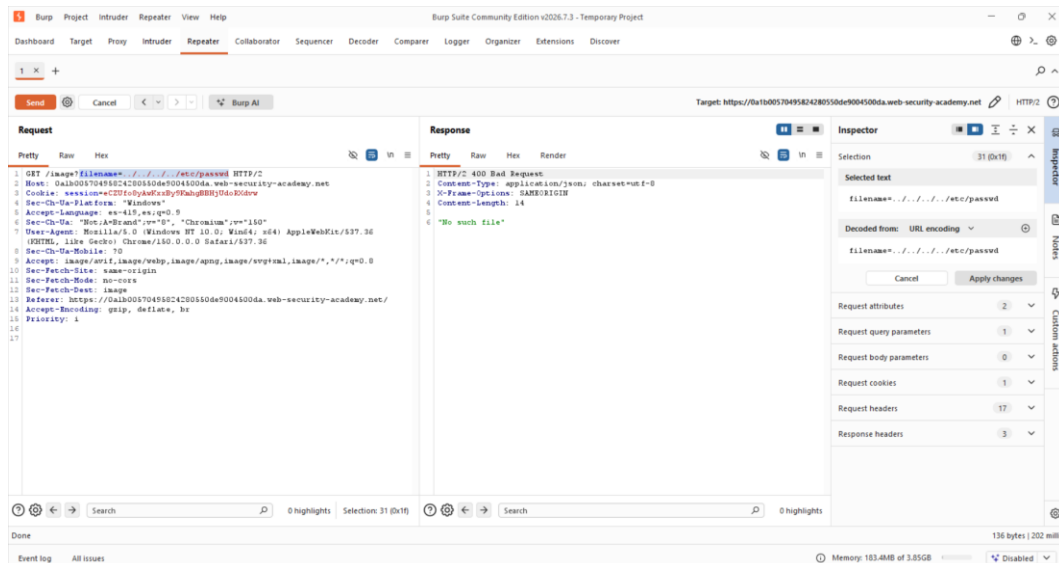


Figura 8. Recepción de un error 400 Bad Request indicando que las secuencias relativas son bloqueadas por el servidor.

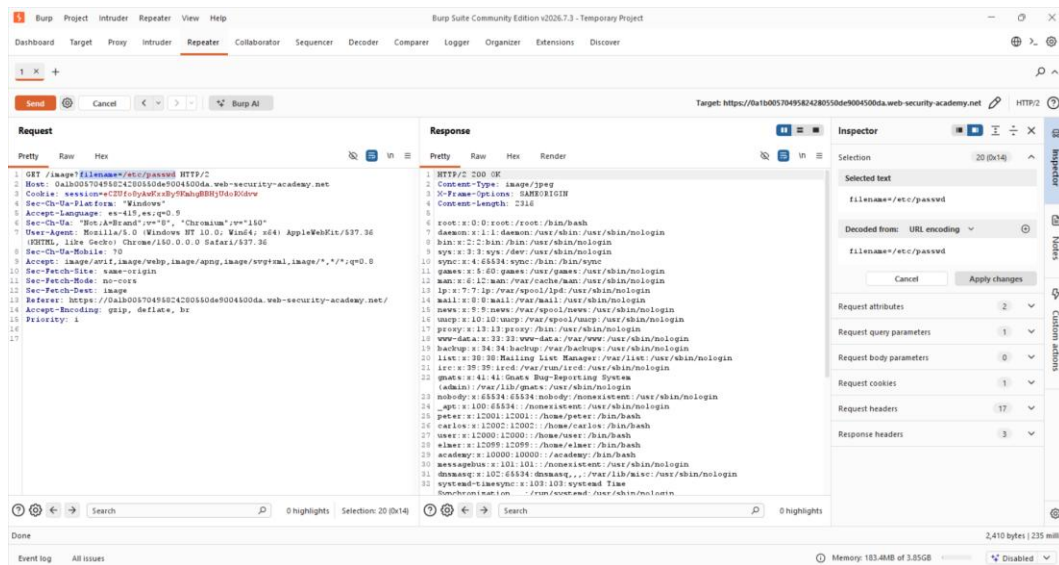


Figura 9. Inyección exitosa de la ruta absoluta /etc/passwd obteniendo el archivo del sistema con código 200 OK.

Evidencias obligatorias

Request interceptado

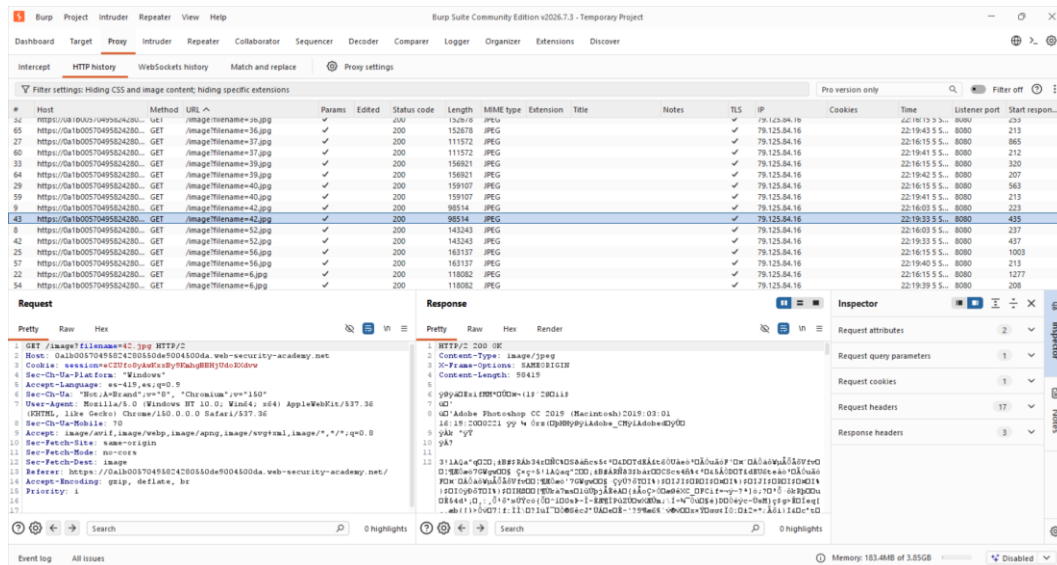


Figura 10. Request interceptado — Petición HTTP GET interceptada en Burp Proxy solicitando el recurso legítimo /image?filename=42.jpg.

Payload utilizado

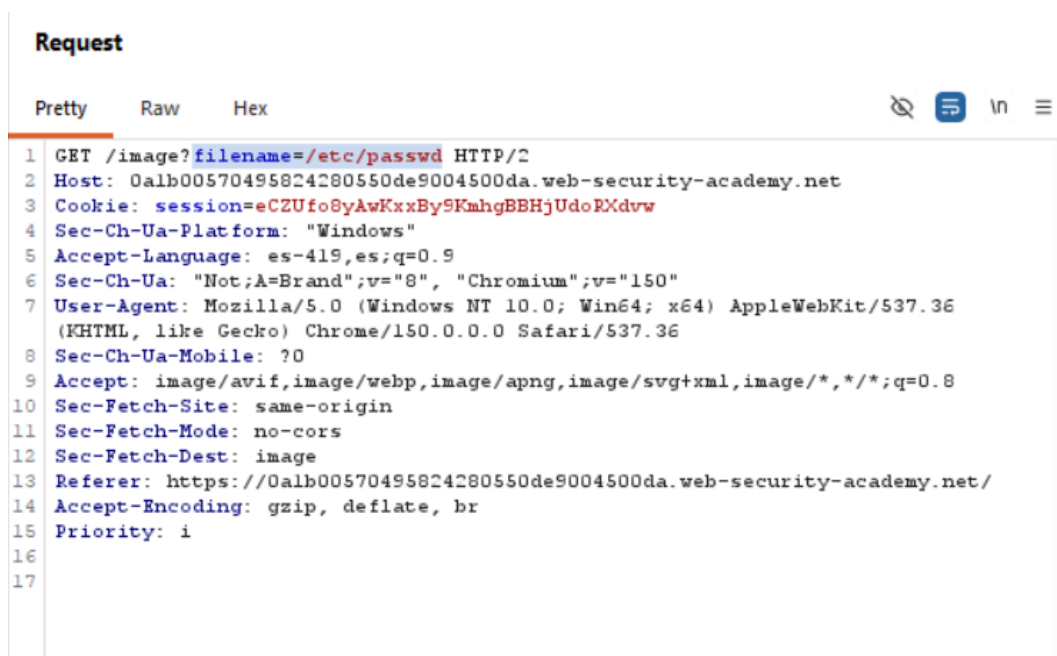


Figura 11. Payload utilizado — Petición HTTP modificada en Repeater inyectando la ruta absoluta /etc/passwd en el parámetro filename.

Respuesta vulnerable

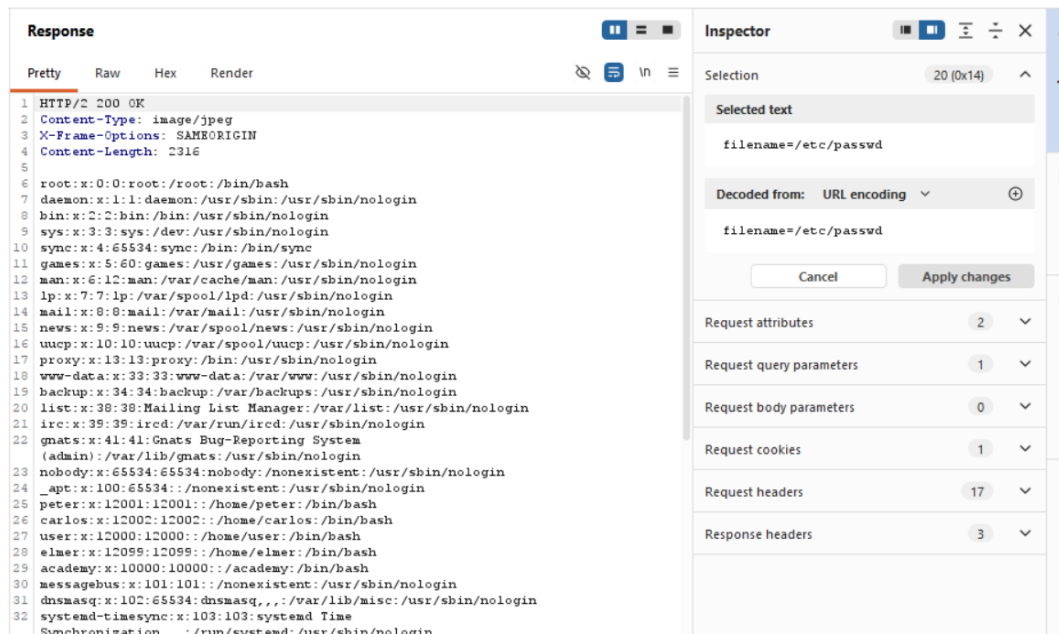


Figura 12. Respuesta vulnerable — Respuesta HTTP 200 OK del servidor retornando en el cuerpo el contenido del archivo sensible /etc/passwd.

Confirmación de “Lab Solved”

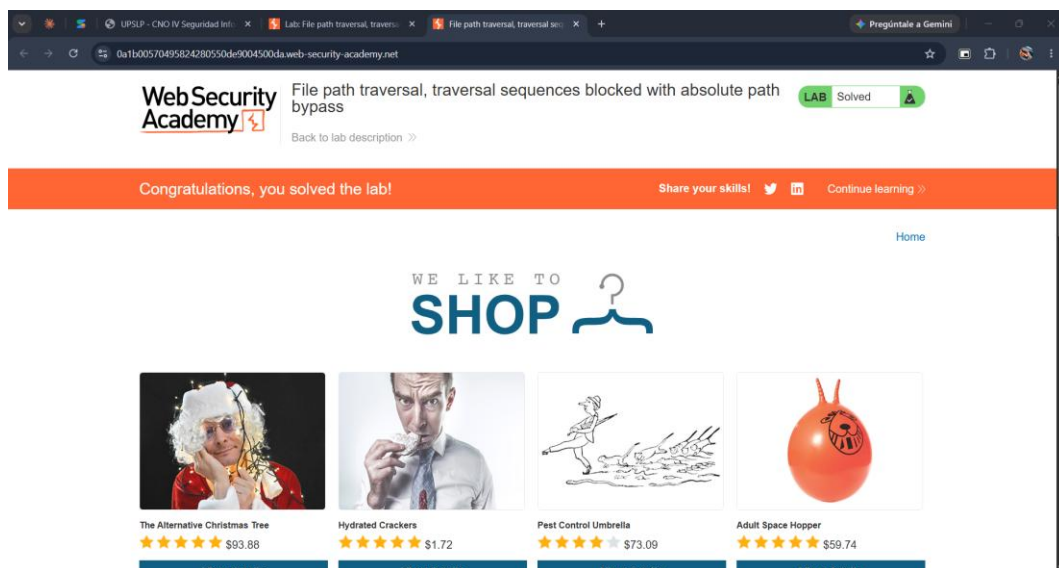


Figura 13. Confirmación de “Lab Solved” — Confirmación en el navegador de la aplicación con la notificación 'Congratulations, you solved the lab!'.

Explicación del payload

El payload definitivo empleado es: /etc/passwd

Análisis sintáctico y lógico:

1. Carácter inicial '/': Representa el directorio raíz (root) del sistema de archivos en entornos tipo Unix/Linux. Al situarlo al comienzo de la cadena, la llamada al sistema de archivos subyacente la interpreta como una ruta absoluta en lugar de un archivo relativo al directorio de trabajo predeterminado.
2. Subdirectorio 'etc': Carpeta estándar del sistema que almacena archivos de configuración y tablas de administración de usuarios.
3. Recurso 'passwd': Archivo que contiene el listado de todas las cuentas locales del sistema operativo, identificadores UID/GID, directorios home y shells asignadas.

Mecanismo de evasión y alteración lógica:

La aplicación cuenta con una validación superficial que verifica y bloquea la presencia de patrones de retroceso de directorio como '../'. Al no utilizar ninguna de estas secuencias y proporcionar directamente una ruta absoluta que comienza con '/', el filtro de seguridad de la aplicación es completamente eludido. La API de lectura en el backend concatena o resuelve la ruta directamente desde la raíz del sistema, anulando la restricción de entorno y entregando el archivo crítico al cliente.

Mitigación recomendada

Para mitigar de forma integral y definitiva esta vulnerabilidad, se debe implementar una estrategia de defensa en profundidad compuesta por:

1. Eliminación del paso directo de rutas al sistema de archivos (Remediación en Código): Evitar que la entrada del usuario interactúe con APIs del sistema operativo. La mejor práctica consiste en almacenar las imágenes con identificadores únicos (IDs numéricos o UUIDs) en una base de datos o almacenamiento de objetos (ej. Amazon S3, Azure Blob), desacoplando totalmente las peticiones del sistema de archivos local.
2. Lista Blanca Estricta y Validación de Entrada: Si es indispensable manejar nombres de archivo, validar mediante una lista blanca de caracteres estrictamente alfanuméricos (ej. `^[a-zA-Z0-9_-]+\.(jpg|png|gif)$`), rechazando de forma categórica caracteres especiales como '/', '\', y secuencias de control.
3. Canonización y Verificación de Ruta Base (Path Canonicalization): Antes de abrir cualquier archivo, se debe resolver la ruta absoluta canónica y validar que el resultado comience estrictamente con el directorio base autorizado. Por ejemplo, en Java:

```
File file = new File(BASE_DIRECTORY, userInput);
if (!file.getCanonicalPath().startsWith(BASE_DIRECTORY)) {
    throw new SecurityException("Acceso no autorizado");
}
```
4. Principio de Menor Privilegio (Arquitectura): Ejecutar el servidor web y los procesos de backend bajo una cuenta de servicio sin privilegios (ej. `www-data` o un usuario dedicado en un contenedor sin permisos de root), restringiendo mediante permisos de sistema de archivos (DAC/MAC) el acceso de lectura únicamente a los directorios estrictamente necesarios.